

บทที่ 3

การออกแบบและพัฒนา

การออกแบบระบบประกอบด้วย การออกแบบ 2 ส่วนด้วยกัน ได้แก่

- 1) การออกแบบทางด้านซอฟต์แวร์
- 2) การออกแบบทางด้านฮาร์ดแวร์

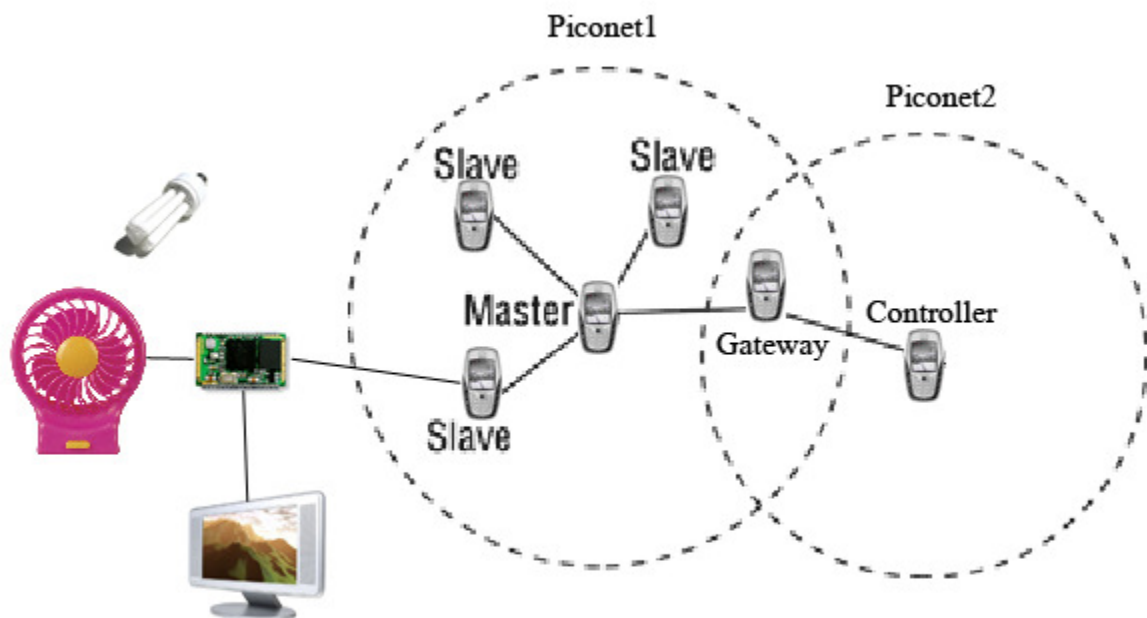
โดยในบทนี้จะขออธิบายถึงการออกแบบระบบ 2 หัวข้อด้วยกัน ได้แก่

- 1) แนวทางการออกแบบระบบ
- 2) ขั้นตอนและวิธีการออกแบบระบบ

3.1 แนวทางการออกแบบระบบ

3.1.1 แนวทางการพัฒนาระบบ

ในโครงการนี้ได้เลือกระบบลูทูลมารับหน้าที่ในการสื่อสารระหว่างโทรศัพท์มือถือและอุปกรณ์ไฟฟ้าในบ้าน ในระบบนี้จะใช้หลักการของ Bluetooth point-to-multipoint ซึ่งจะเป็นการใช้เทคโนโลยีบลูทูธในระดับสูงเพื่อสร้างการเชื่อมต่อแบบ point-to-multipoint โดยระบบจะประกอบด้วย 6 ส่วนดังที่จะกล่าวในส่วนต่อไป



รูปที่ 3.1 แสดงภาพรวมของการออกแบบในโครงการนี้

ส่วนประกอบที่อยู่ในระบบดังรูปที่ 3.1 ประกอบไปด้วย

1) มาสเตอร์ สามารถใช้โทรศัพท์มือถือที่มีระบบปฏิบัติการซิมเบียน หรือ Bluetooth module ซึ่งในโครงงานนี้จะใช้โทรศัพท์มือถือเป็นตัว มาสเตอร์ ซึ่งเป็นอุปกรณ์ที่สร้างการเชื่อมต่อไปยังอุปกรณ์อื่นในที่นี้คือสเลฟ มาสเตอร์ จะค้นหาอุปกรณ์สเลฟและ Service ของแต่ละสเลฟ โดยสามารถสร้างการเชื่อมต่อไปยังหลายๆสเลฟและรักษาการเชื่อมต่อให้ทำงานได้พร้อมๆ กัน กลายเป็นเครือข่าย Piconet ขึ้นมา โดยหน้าที่หลักของ มาสเตอร์ คือ

- 1.1 เป็นตัวสร้างการเชื่อมต่อในแต่ละ Piconet (แต่ละ Piconet จะมี มาสเตอร์ เดียว)
- 1.2 ส่งคำสั่งไปให้กับสเลฟทุกตัวสเลฟบางตัว หรือ เกตเวย์
- 1.3 รอรับการเชื่อมต่อจาก คอนโทรลเลอร์ ในกรณีที่ คอนโทรลเลอร์ ต้องการเชื่อมต่อกับ มาสเตอร์ โดยตรง (อยู่ในระยะของ มาสเตอร์)

2) สเลฟสามารถใช้โทรศัพท์มือถือที่มีระบบปฏิบัติการซิมเบียน หรือ Bluetooth Module เช่นเดียวกับมาสเตอร์ แต่สเลฟไม่สามารถสร้างการเชื่อมต่อใดๆ ได้ มันมีหน้าที่เพียงเป็นตัวรับฟังรอรับการเชื่อมต่อจากอุปกรณ์ มาสเตอร์ เมื่อสร้างการเชื่อมต่อกับ มาสเตอร์ เป็น Piconet แล้ว จะเปิดช่องสัญญาณเพื่อรับการเชื่อมต่อจาก คอนโทรลเลอร์ หรือ มาสเตอร์ ตัวอื่นต่อไป ซึ่งอุปกรณ์สเลฟจะมีหน้าที่หลักคือ

- 2.1 รับคำสั่งจาก มาสเตอร์ จากนั้นนำคำสั่งไปประมวลผลแล้วส่งสัญญาณไปยังวงจรควบคุมอุปกรณ์ไฟฟ้าภายในบ้าน
- 2.2 ทำหน้าที่เป็น เกตเวย์ เชื่อมต่อระหว่าง Piconet เพื่อสร้างเครือข่าย Scatternet
- 2.3 รับคำสั่งจาก คอนโทรลเลอร์ ในนำคำสั่งไปประมวลผล ในกรณีที่เป็นการคำสั่งของตัวเองหรือส่งต่อไปยัง มาสเตอร์ เพื่อส่งต่อไปยังสเลฟตัวอื่น

3) คอนโทรลเลอร์ เป็นอุปกรณ์ที่เป็นมีฟังก์ชันการทำงานเหมือน มาสเตอร์ แต่จะมี User Interface ที่สวยงามเพื่ออำนวยความสะดวกในการส่งคำสั่งควบคุม โดยหน้าที่หลักของคอนโทรลเลอร์ คือ

- 3.1 สร้างการเชื่อมต่อเป็น Piconet กับ เกตเวย์ เพื่อส่งคำสั่งผ่าน ไปยัง มาสเตอร์ หรือ ควบคุมสเลฟที่เป็น เกตเวย์ โดยตรง
- 3.2 สร้างการเชื่อมต่อกับ มาสเตอร์ โดยตรงเพื่อควบคุมอุปกรณ์ไฟฟ้าทุกตัวได้ผ่าน มาสเตอร์
- 3.3 สร้างการเชื่อมต่อกับ เกตเวย์ ในกรณีที่ คอนโทรลเลอร์อยู่นอกระยะของ มาสเตอร์ เพื่อส่งคำสั่งผ่าน เกตเวย์ ไปยัง มาสเตอร์
- 3.4 ควบคุมสเลฟแต่ละตัวโดยตรง

4) เกตเวย์ คือสเลฟที่มีหน้าที่ในการสร้างการเชื่อมต่อและส่งคำสั่งไปมาระหว่าง มาสเตอร์ กับ มาสเตอร์ หรือ มาสเตอร์ กับ คอนโทรลเลอร์ ซึ่ง เกตเวย์ จะมีคุณสมบัติเหมือน

เสาทุกประการ คือสามารถต่อกับวงจรควบคุมเครื่องใช้ไฟฟ้าได้ โดยจะเพิ่มหน้าที่สำคัญคือเป็นตัวเชื่อมต่อระหว่าง Piconet

5) วงจรควบคุมเครื่องใช้ไฟฟ้า โดยรับสัญญาณจากเสาจากนั้นนำสัญญาณไปประมวลผลด้วยไมโครคอนโทรลเลอร์ MSC-51 ซึ่งจะควบคุมรีเลย์เพื่อควบคุมเครื่องใช้ไฟฟ้า โดยแต่ละวงจรสามารถควบคุม เครื่องใช้ไฟฟ้าได้หลายตัว (รายละเอียดของวงจรควบคุมอุปกรณ์ไฟฟ้าจะกล่าวในหัวข้อต่อไป

6) เครื่องใช้ไฟฟ้า

3.2 ขั้นตอนและวิธีการออกแบบระบบ

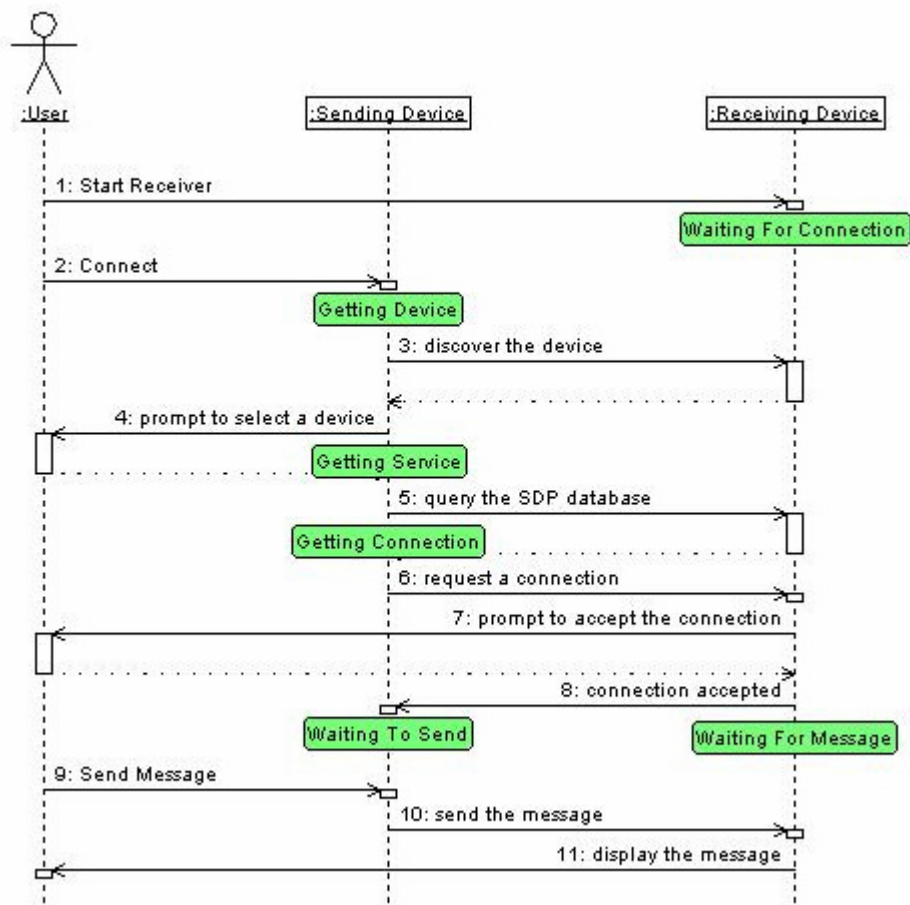


รูปที่ 3.2 แผนภาพขั้นตอนการออกแบบระบบ

3.2.1 ศึกษาและเขียนโปรแกรมเพื่อทดสอบการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่สองเครื่องแบบ Point-to-Point ผ่านการสื่อสารไร้สายแบบบลูทูธ

ในขั้นตอนนี้ ได้ทำการศึกษาในเรื่องการติดต่อผ่านระบบสื่อสารไร้สายบลูทูธโดยใช้โทรศัพท์มือถือที่ใช้ระบบปฏิบัติการซิมเบียน 2 เครื่องในการทดสอบ โดยออกแบบให้ใช้ภาษาซิมเบียนซีพลัสพลัส (Symbian C++) ในการเขียนโปรแกรมทดสอบขึ้นมา

จากการศึกษาการสื่อสารทางด้านการบลูทูธ ทำให้พบว่าอุปกรณ์บลูทูธแต่ละชิ้นจะมีค่าซึ่งไม่ซ้ำกัน เรียกว่า Bluetooth Device Address เป็นเลขฐานสิบหกจำนวน 12 หลักด้วยกัน การสื่อสารระหว่างอุปกรณ์บลูทูธสองชิ้นจะต้องอ้างอิงถึงค่านี้เสมอ เช่นเดียวกับการสื่อสารผ่านบลูทูธระหว่างโทรศัพท์มือถือซิมเบียนสองเครื่อง แต่จะต่างกันตรงที่การสื่อสารระหว่างโทรศัพท์มือถือซิมเบียนตัวระบบปฏิบัติการจะทำการหา Bluetooth Device Address ของอุปกรณ์รอบข้างให้โดยอัตโนมัติ โดยเราไม่จำเป็นต้องจำค่าเหล่านี้ได้เลย สำหรับตัวโปรแกรมจะออกแบบให้สามารถติดต่อส่งข้อความระหว่างโทรศัพท์มือถือสองเครื่องได้ จะมีขั้นตอนการทำงานหลังจากกดเมนูเพื่อทำในคำสั่งต่างๆ จะแสดงได้ดังแผนภาพในรูปที่ 3.3

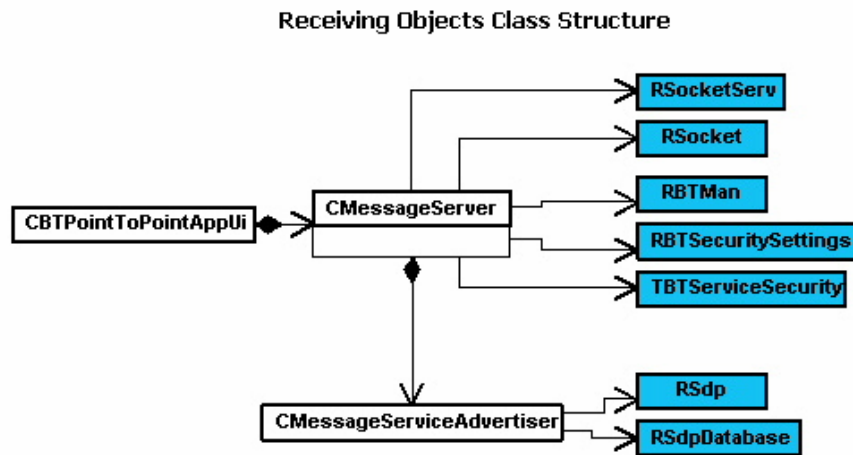


รูปที่ 3.3 ขั้นตอนการติดต่อบลูทูธของซิมเบียน

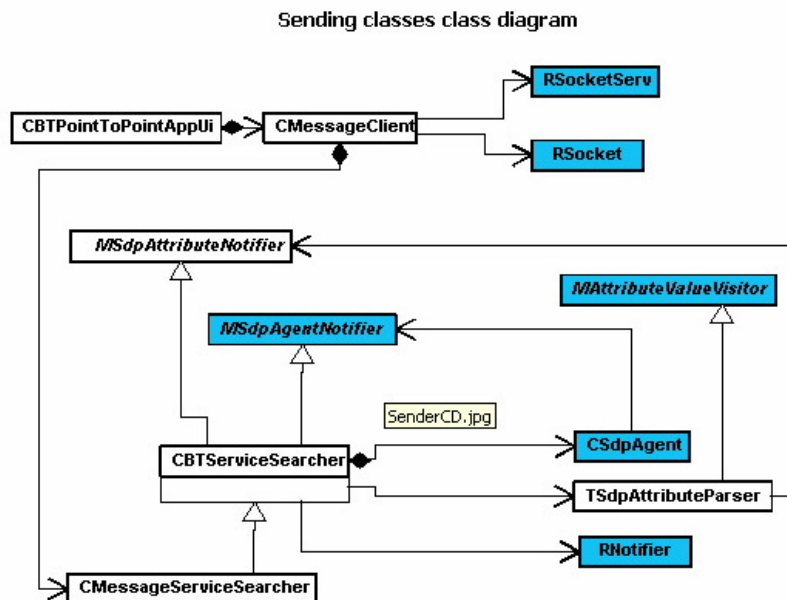
ตารางที่ 3.1 ขั้นตอนต่างๆในการติดต่อบลูทูธ

ขั้นตอน	ความหมาย
1. Start Receiver	ใช้ทำให้โทรศัพท์มือถือทำหน้าที่เป็นตัวรับ
2. Connect	ผู้ใช้กดเลือกที่จะให้โทรศัพท์มือถือติดต่อไปยังเครื่องปลายทางที่เปิดบลูทูธ
3. Discover the device	ตรวจสอบหาเครื่องที่เปิดบลูทูธไว้ในรัศมีที่เครื่องสามารถหาเจอได้
4. Prompt to select a device	ผู้ใช้เลือกเครื่องปลายทางที่จะติดต่อไป ซึ่งจะขึ้นเป็นรายการมาให้เลือก
5. Query the SDP database	รับข้อมูลต่างๆ ของเครื่องที่ผู้ใช้ได้เลือก จากฐานข้อมูลเอสดีพี (SDP Database)
6. Request a connection	โทรศัพท์มือถือส่งไปขอการติดต่อผ่านระบบสื่อสารไร้สายบลูทูธ
7. Prompt to accept the connection	ใช้สำหรับโทรศัพท์มือถือส่งไปขอการติดต่อผ่านระบบสื่อสารไร้สายบลูทูธ
8. Connection accepted	เป็นเหตุการณ์ที่บ่งบอกว่าเครื่องรับยอมรับการติดต่อแล้ว
9. Send Message	เป็นคำสั่งในเมนู ไว้สำหรับเครื่องส่งที่ต้องการส่งข้อมูลไปยังเครื่องปลายทาง
10. Send the message	เครื่องส่งข้อความไปยังเครื่องปลายทาง
11. Display the message	แสดงผลจากการส่งข้อความบนหน้าจอ

ในขั้นตอนนี้เราได้ศึกษาและเขียนโปรแกรมเพื่อทดสอบการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่สองเครื่องแบบ Point-to-Point ผ่านการสื่อสารไร้สายแบบบลูทูธ ซึ่งเป็นพื้นฐานสำคัญในการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่สองเครื่องแบบ Point-to-Multipoint ต่อไปสำหรับคลาส สำคัญๆที่ใช้ในการรับส่งข้อมูลได้แสดงเป็น Class Diagram ดังนี้



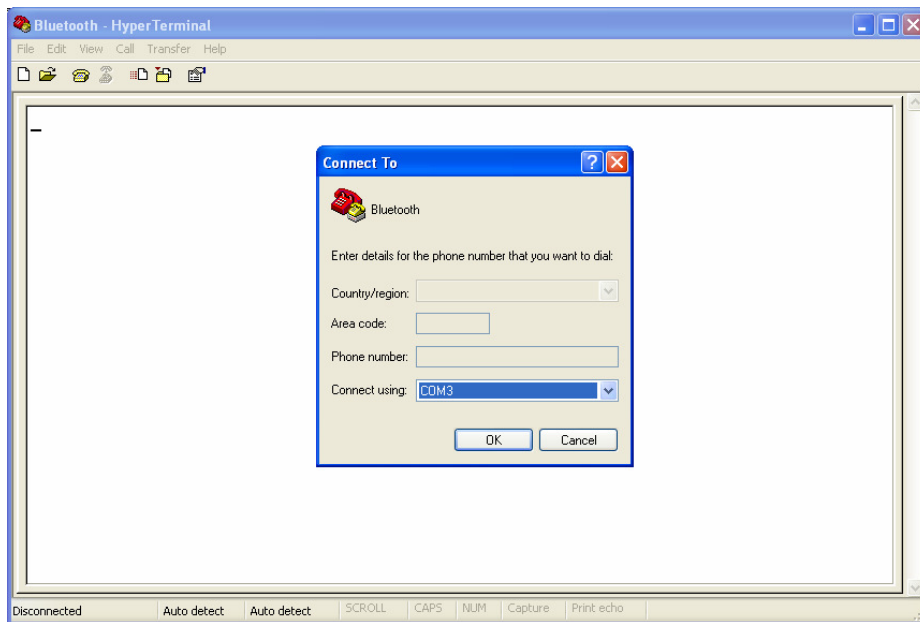
รูปที่ 3.4 Class Diagram สำหรับการรับข้อมูล



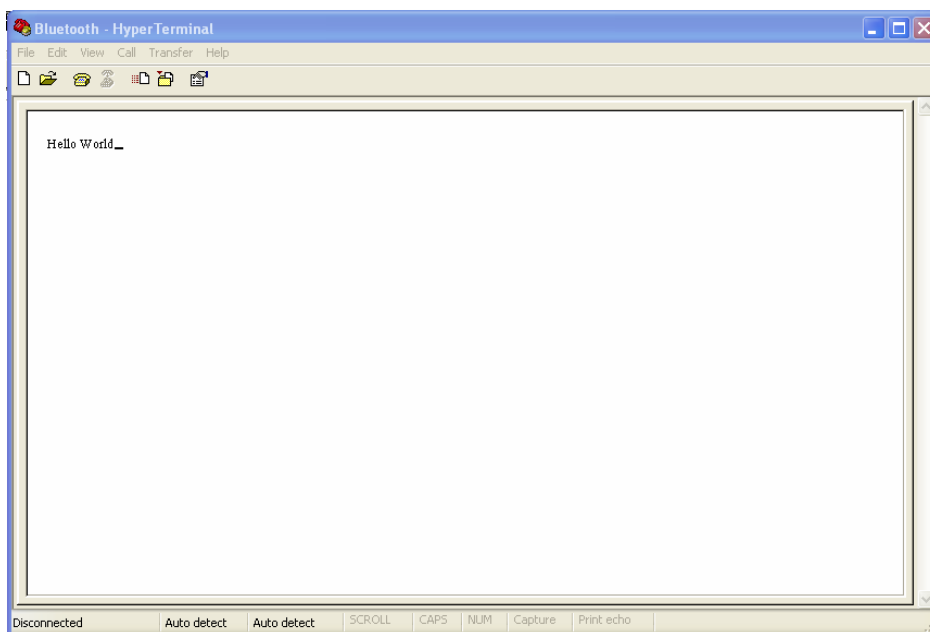
รูปที่ 3.5 Class Diagram สำหรับการส่งข้อมูล

3.2.2 ศึกษาและทดสอบการรับสัญญาณจากอุปกรณ์บลูทูธเข้าคอมพิวเตอร์เพื่อทำการทดสอบการสื่อสารระหว่างโทรศัพท์เคลื่อนที่กับคอมพิวเตอร์

ในขั้นตอนนี้จะใช้คอมพิวเตอร์ที่ต่อ Bluetooth Module Adapter ที่ port USB เป็นตัวคอยรับการเชื่อมต่อและรับข้อความจากโทรศัพท์มือถือเป็นเครื่องส่ง ซึ่งคอมพิวเตอร์จะรอรับการเชื่อมต่อที่ port COM3 โดยใช้ Hyper Terminal และโทรศัพท์มือถือจะทำการเชื่อมต่อมาโดยใช้ port COM4 โดยใช้โปรแกรม btpointtopoint



รูปที่ 3.6 โปรแกรม Hyper Terminal เปิดรอรับการเชื่อมต่อที่ port COM3



รูปที่ 3.7 ข้อความที่ได้รับจากโทรศัพท์มือถือผ่านบลูทูธ

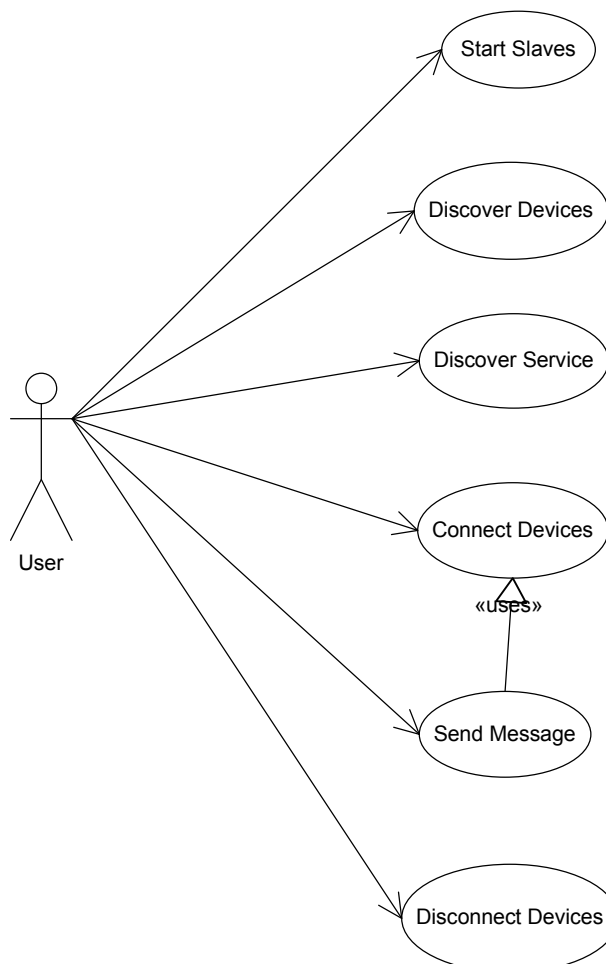
การศึกษาและทดสอบการรับสัญญาณจากอุปกรณ์บลูทูธเข้าคอมพิวเตอร์เพื่อทำการทดสอบการสื่อสารระหว่างโทรศัพท์เคลื่อนที่กับคอมพิวเตอร์ ซึ่งจากนั้นเราจะได้ออกแบบสร้างวงจรควบคุมอุปกรณ์ไฟฟ้าอย่างง่ายบนคอมพิวเตอร์ต่อไป

3.2.3 ศึกษาและเขียนโปรแกรมเพื่อทดสอบการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่หลายเครื่องแบบ Point-to-Multipoint ผ่านการสื่อสารไร้สายแบบบลูทูธ ในเครือข่าย piconet

หลังจากที่ได้ศึกษาการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่สองเครื่องแบบ Point-to-Point ผ่านการสื่อสารไร้สายแบบบลูทูธแล้ว ทำให้เข้าใจหลักการทำงานของบลูทูธซึ่งต่อไปจะได้ทำการศึกษาและเขียนโปรแกรมเพื่อทดสอบการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่หลายเครื่องแบบ Point-to-Multipoint ซึ่งถือเป็นตัวหลักของโครงงานนี้

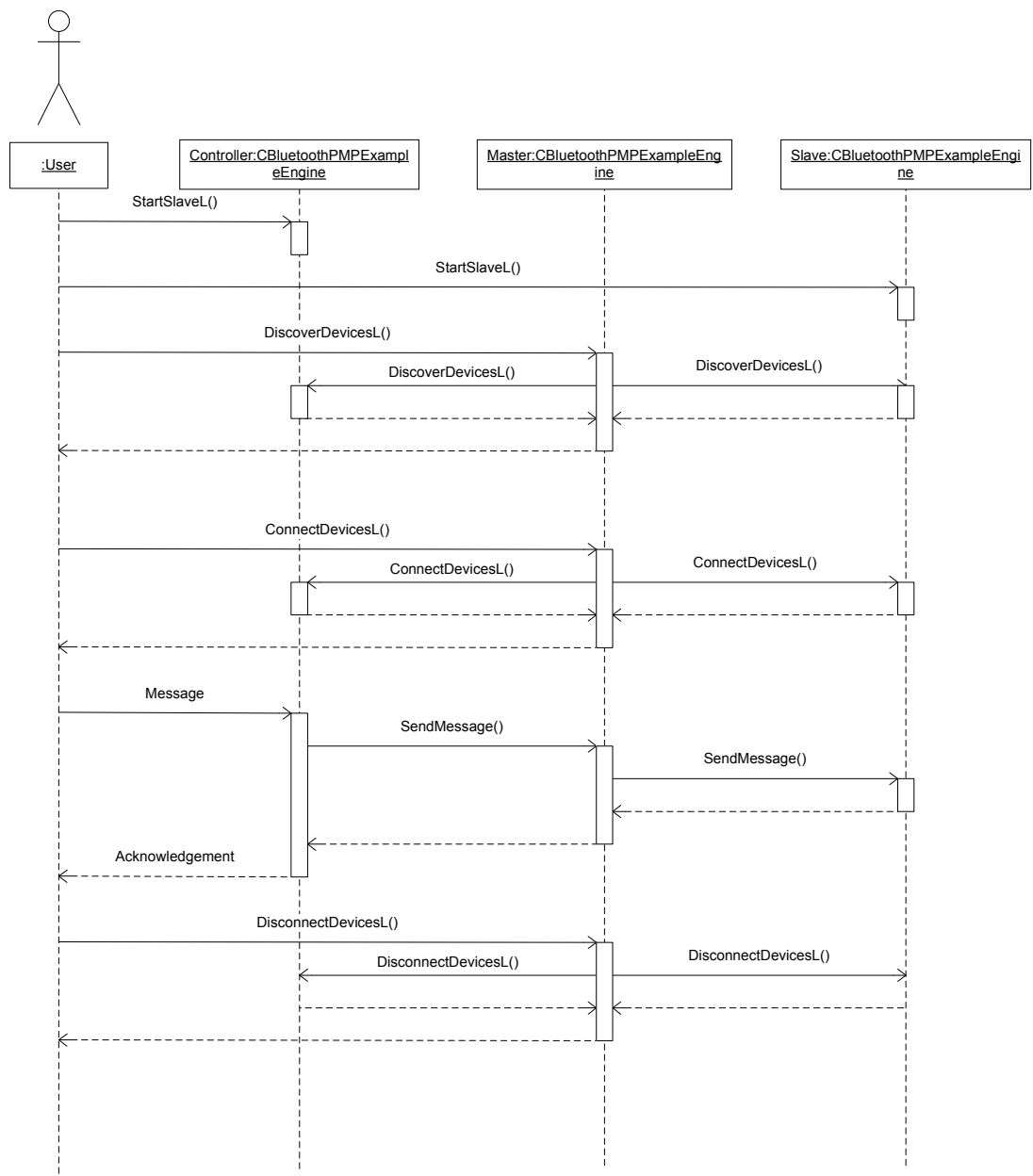
โปรแกรม BluetoothPMP ที่ได้ทำการพัฒนาขึ้นมาเป็นโปรแกรมระดับสูงที่แสดงการใช้งานของเทคโนโลยีบลูทูธในการสร้างการเชื่อมต่อแบบ Point-to-Multipoint เพื่อสร้างเป็นเครือข่าย piconet ของโทรศัพท์เคลื่อนที่โดยติดต่อสื่อสารกันผ่านบลูทูธ

Use-Case diagram

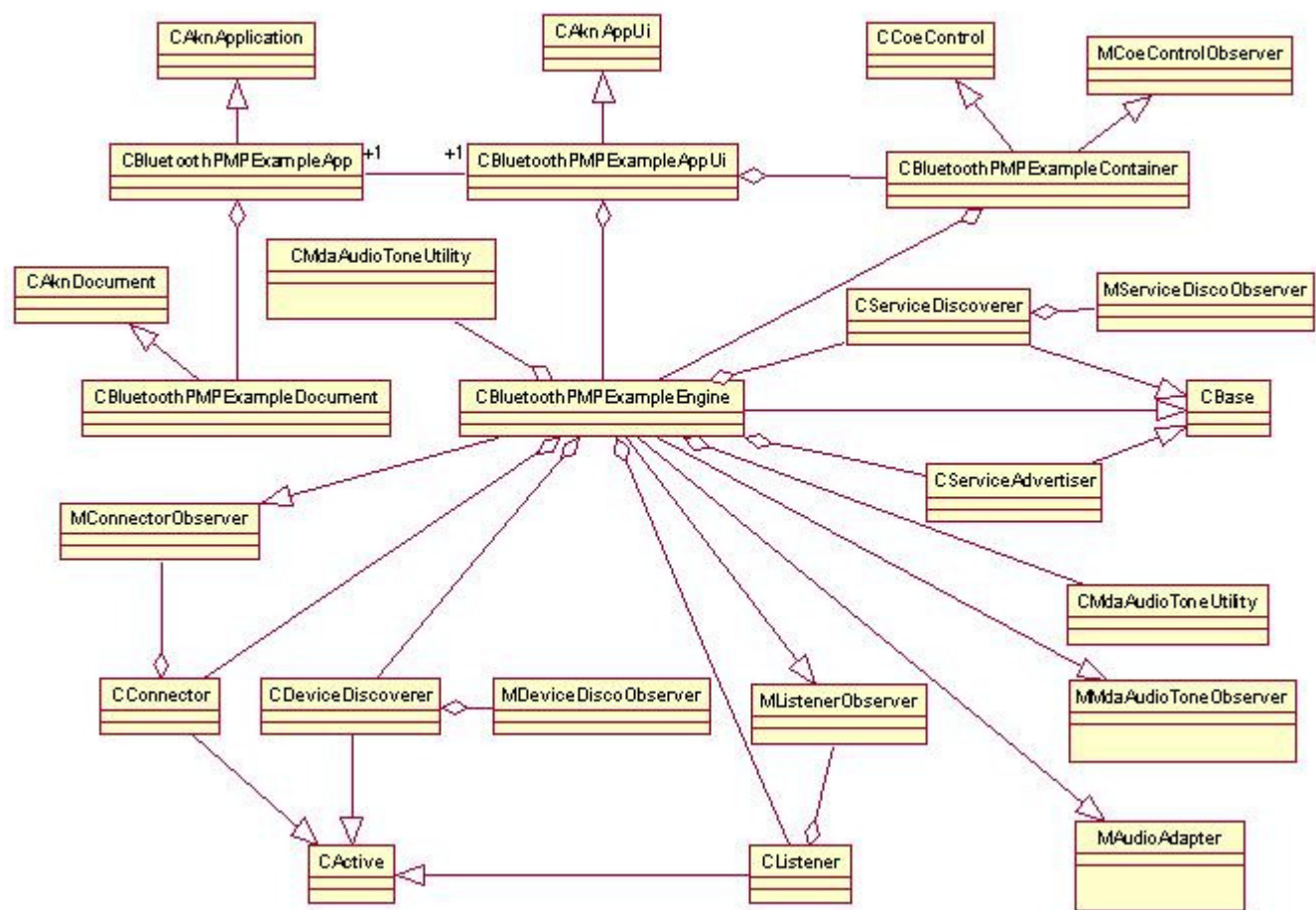


รูปที่ 3.8 Use-Case Diagram

Sequence Diagram



รูปที่ 3.9 Sequence Diagram การทำงานอย่างง่าย (เพื่อเสริมความเข้าใจ)



รูปที่ 3.10 Class Diagram

รายละเอียดของคลาสต่างๆใน โปรแกรม BluetoothPMP ที่สำคัญมีดังนี้

- **CBluetoothPMPExampleApp** ประกาศเป็นคลาสหลักของโปรแกรม และประกาศ UID ของโปรแกรม

```
const TUid KUidBluetoothPMPExample = { 0x101FF1C5 };
```

- **CBluetoothPMPExampleAppUi** ประกาศเป็นคลาส User Interface สำหรับติดต่อกับผู้ใช้ มีหน้าที่สำหรับจัดการกับการแสดงเมนูในโปรแกรมและคำสั่งสำหรับแต่ละเมนู

คำสั่งในการแสดงเมนู

```
aMenuPane->SetItemDimmed(EBTCmdDiscoverDevices, EFalse);
```

หมายเหตุ

EFalse = แสดง

ETrue = ไม่แสดง

ฟังก์ชันในการจัดการกับ Event เมื่อ User เลือกเมนูต่างๆ

```
void CBluetoothPMPExampleAppUi::HandleCommandL(TInt aCommand)
```

```
{
    switch ( aCommand )
    {
        case EAknSoftkeyBack:
        case EEikCmdExit:
        {
            Exit();
            break;
        }

        case EBTCmdStartเสิร์ฟ:
        {
            if ( !iEngine->Isเสิร์ฟ )
                iEngine->Startเสิร์ฟL();
            break;
        }
    }
}
```

```

        }
        ...
    default:
        break;
    }
}

```

- **CBluetoothPMPEExampleDocument** ประกาศเอกสารของโปรแกรม
- **CBluetoothPMPEExampleContainer** ประกาศ container control ของโปรแกรม เป็นตัวสร้าง window ของโปรแกรมขึ้นมาโดย

```

void CBluetoothPMPEExampleContainer::ConstructL(const TRect& aRect)
{
    // create new window,
    CreateWindowL();

    // create label to display status to user
    iLabel = new (ELeave) CEikLabel;
    iLabel->SetContainerWindowL( *this );
    iLabel->SetFont( LatinBold12() );

    // set window size
    SetRect(aRect);

    _LIT(KInfo, "Bluetooth Remote คอนโทรลเลอร์");
    iLabel->SetTextL( KInfo );

    // activate window
    ActivateL();
}

```

- **CBluetoothPMPEExampleEngine** เป็นคลาสที่เก็บฟังก์ชัน และ event callback ที่สำคัญๆ เพื่อที่จะให้โปรแกรมเรียกใช้งาน ตัวอย่างฟังก์ชันที่สำคัญได้แก่

ค้นหาอุปกรณ์บลูทูธในระยะ อุปกรณ์ที่ถูกค้นพบจะแสดงต่อผู้ใช้

```
void DiscoverDevicesL();
```

เริ่มค้นหา Service บนอุปกรณ์อื่น

```
void DiscoverServicesL();
```

เริ่มโปรแกรมในสแตฟmode โปรแกรมจะเปิด Sockets รอรับความต้องการ
การเชื่อมต่อ และ Service ของมัน

```
void StartสแตฟL();
```

ส่งข้อความไปยังสแตฟทุกตัว ผู้ใช้สามารถใส่ข้อความที่ต้องการส่ง

```
void SendMessage();
```

เชื่อมต่อไปยังอุปกรณ์อื่นหลังจากพบ Service ที่เราต้องการ

```
void ConnectDevicesL();
```

ยกเลิกการเชื่อมต่อกับอุปกรณ์อื่น

```
void DisconnectDevicesL();
```

แสดงอุปกรณ์ที่ทำการเชื่อมต่อ

```
void ShowConnectedDevicesL();
```

จัดการข้อมูลที่สแตฟ ได้รับจาก มาสเตอร์

```
void HandleListenerDataReceivedL(TDesC& aData);
```

จัดการข้อมูลที่ มาสเตอร์ ได้รับจาก สแตฟ

```
void HandleConnectorDataReceivedL(THostName aName, TDesC&  
aData);
```

จัดการเหตุการณ์การเชื่อมต่อของ สแตฟ

```
void HandleListenerConnectedL();
```

จัดการเหตุการณ์การยกเลิกการเชื่อมต่อของ เซลล์

```
void HandleListenerDisconnectedL();
```

จัดการเหตุการณ์ที่การค้นหาอุปกรณ์สำเร็จ

```
void HandleDeviceDiscoveryCompleteL();
```

จัดการเหตุการณ์ที่การค้นหา Service ของอุปกรณ์สำเร็จ

```
void HandleServiceDiscoveryCompleteL();
```

คืนค่าเป็นจริงถ้า มาสเตอร์ มีการเชื่อมต่อกับ เซลล์

```
TBool HasConnections();
```

แสดงข้อความ

```
void ShowMessageL(const TDesC& /* aMsg */, TBool /* aReset=false */);
```

- **CServiceDiscoverer** เป็นคลาสที่ใช้ในการค้นหา Service ที่ต้องการบนอุปกรณ์บลูทูธ ใช้กับอุปกรณ์ที่เป็น มาสเตอร์ ถูกเรียกใช้โดย instance ของคลาส CBluetoothPMPEngine
- **CServiceAdvertiser** เป็นคลาสที่ใช้ Service ของตนเอง ใช้กับอุปกรณ์ที่เป็น เซลล์ถูกเรียกใช้โดย instance ของคลาส CBluetoothPMPEngine
- **CDeviceDiscoverer** เป็นคลาสที่ใช้ค้นหาอุปกรณ์ที่มีบลูทูธและอยู่ในระยะ ถูกเรียกใช้โดย instance ของคลาส CBluetoothPMPEngine
- **CConnector** เป็นคลาสที่เก็บฟังก์ชันและคุณสมบัติของตัว Connector ซึ่งจะประกาศไว้ใน Constructor ได้แก่ HostName, Address, Port เป็นต้นซึ่งจะนำไปใช้สร้าง Socket สำหรับ มาสเตอร์ เพื่อเชื่อมต่อไปยัง เซลล์

```

TRequestStatus CConnector::ConnectL(THostName aName, TBTDDevAddr aAddr,
TInt aPort)
{
    iName=aName;
    iAddr=aAddr;
    iPort=aPort;

    // load protocol, RFCOMM
    TProtocolDesc pdesc;
    User::LeaveIfError(iSocketServ->FindProtocol(_L("RFCOMM"), pdesc));

    // open socket
    User::LeaveIfError(iSock.Open(*iSocketServ, _L("RFCOMM")));

    // set address and port
    TBTSockAddr addr;
    addr.SetBTAddr(iAddr);
    addr.SetPort(iPort);

    // connect socket
    TRequestStatus status;
    iSock.Connect(addr, status);
    User::WaitForRequest(status);
    if ( status!=KErrNone )
    {
        // error opening conn
        return status;
    }
    iState=EConnecting;
    WaitAndReceiveL();
    return status;
}

```

- **CListener** เป็นคลาสที่เก็บฟังก์ชันและคุณสมบัติของตัว Listener ในเซิร์ฟซึ่ง จะทำการเปิด Socket ใ้รอฟังรับการเชื่อมต่อจาก มาสเตอร์ ฟังก์ชันที่สำคัญได้แก่

เริ่มรอรับการเชื่อมต่อ

```
TInt StartListenerL();
```

หยุดรอรับการเชื่อมต่อ

```
void StopListenerL();
```

- **struct TDeviceData** เป็น Struct ที่ประกาศองค์ประกอบที่อยู่ใน TDeviceData ได้แก่

```
struct TDeviceData  
{  
    THostName iDeviceName;  
    TBTDevAddr iDeviceAddr;  
    TUInt iDeviceServicePort;  
};
```

- **MAudioAdapter** เป็น คลาส ที่เกี่ยวกับสั่งงานให้โทรศัพท์เคลื่อนที่ส่งสัญญาณเสียงออกมา เป็นคลาส ที่ คลาส CBluetoothPMPEngine จะ inherit มาใช้งาน ภายในคลาสจะประกาศฟังก์ชันไว้เป็น virtual เพื่อให้ CBluetoothPMPEngine ทำการ Override ฟังก์ชันเหล่านั้นมาใช้งาน ฟังก์ชันที่สำคัญได้แก่

```
virtual void PlayL() = 0;  
virtual void StopL() = 0;  
virtual void RecordL() = 0;
```

- **MMdaAudioToneObserver** เป็น คลาส ที่เกี่ยวกับสั่งงานให้ โทรศัพท์เคลื่อนที่ส่งสัญญาณเสียงออกมา เป็นคลาส ที่ คลาส CBluetoothPMPEngine จะinherit มาใช้งาน ภายในคลาสจะประกาศ ฟังก์ชันไว้เป็น virtual เพื่อให้ CBluetoothPMPEngine ทำการ Override ฟังก์ชันเหล่านั้นมาใช้งาน ฟังก์ชันที่สำคัญได้แก่

```
virtual void MatoPrepareComplete(TInt aError) = 0;  
virtual void MatoPlayComplete(TInt aError) = 0;
```

- **CMdaAudioToneUtility** เป็นคลาสหลักที่ใช้ในการสั่งงานโทรศัพท์เคลื่อนที่ ให้ส่งสัญญาณเสียงออกมา ในคลาสที่จะนำมาใช้งาน ต้อง inherit สองคลาส

ข้างต้นก่อน โดยคลาส CBluetoothPMPEngine จะสร้าง Object ของ คลาสนี้ เพื่อใช้สั่งงานให้โทรศัพท์เคลื่อนที่ส่งสัญญาณเสียงออกมาเป็นความถี่ ต่างๆ ตามต้องการ สำหรับฟังก์ชันในการใช้งานที่สำคัญได้แก่

ฟังก์ชันยกเลิกการส่งสัญญาณเสียง

```
virtual void CancelPlay() = 0;
```

ฟังก์ชันเริ่มการส่งสัญญาณเสียง

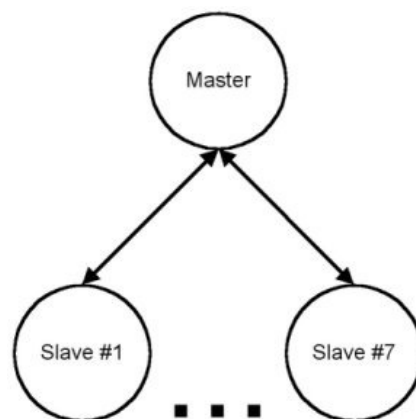
```
virtual void Play() = 0;
```

ฟังก์ชันตั้งค่าการส่งสัญญาณเสียง

```
virtual void PrepareToPlayTone(TInt aFrequency, const  
TTimeIntervalMicroSeconds& aDuration) = 0;
```

ฟังก์ชันตั้งค่าความดังการส่งสัญญาณเสียง

```
virtual void SetVolume(TInt aVolume) = 0;
```



รูปที่ 3.11 เครือข่าย Piconet

จากทฤษฎีในบทที่ 2 เกี่ยวกับการติดต่อสื่อสารในเครือข่าย piconet ซึ่งจะประกอบด้วย อุปกรณ์หลักสองตัวคือ มาสเตอร์ และสเลฟซึ่งจะทำการเชื่อมต่อโดยใช้โปรแกรม BluetoothPMP ที่ได้พัฒนาขึ้นมา จาก คลาส และฟังก์ชันต่างๆที่ได้กล่าวข้างต้น นำมาอธิบายหลักการทำงานของ โปรแกรมได้ดังนี้

1. มาสเตอร์

มาสเตอร์ เป็นอุปกรณ์ที่สร้างการเชื่อมต่อไปยังอุปกรณ์สเลฟเพื่อติดต่อสื่อสารกันผ่านบลูทูธ เพื่อสร้างเป็นเครือข่าย piconet โดยโปรแกรมในอุปกรณ์ มาสเตอร์ ทำงานดังนี้

- มาสเตอร์ จะทำการค้นหาอุปกรณ์บลูทูธที่อยู่ในระยะ โดยใช้ Object ของ คลาส *CDeviceDiscoverer* iDeviceDiscoverer;* เพื่อนำไปค้นหาด้วยคำสั่ง *iDeviceDiscoverer->DiscoverDevicesL(&iDevDataList);*
- เมื่อพบอุปกรณ์บลูทูธในระยะแล้วก็จะใช้ Object ของ Class *CServiceDiscoverer* iServiceDiscoverer;* เพื่อค้นหา Service ที่ต้องการจากอุปกรณ์ที่พบด้วยคำสั่ง *iServiceDiscoverer->DiscoverServicesL(&iDevDataList);*
- เมื่อพบอุปกรณ์ สเลฟ ที่ให้ Service ที่เราต้องการแล้วมาสเตอร์ ก็จะทำการเชื่อมต่อไปยังสเลฟเหล่านั้นโดยจะติดต่อแบบ Point-to-Multipoint สร้างเป็นเครือข่าย Piconet คำสั่งที่ใช้ในการเชื่อมต่อคือ *void CBluetoothPMPEExampleEngine::ConnectDevicesL();* ซึ่งจะไปเรียกใช้ Object และคำสั่งของ คลาส *CConnector*
*CConnector *connector;*
connector = CConnector::NewL(this, &iSocketServ);
connector->ConnectL(dev -> iDeviceName, dev-> iDeviceAddr,dev -> iDeviceServicePort));
- เมื่อทำการเชื่อมต่อแล้ว มาสเตอร์ จะสามารถส่งข้อความไปยังสเลฟทุกตัว หรือ บางตัวได้ โดยใช้ฟังก์ชัน *void CBluetoothPMPEExampleEngine::SendMessage();*
- หากต้องการยกเลิกการเชื่อมต่อกับสเลฟทุกตัว ก็สามารถทำการหยุดการเชื่อมต่อได้โดยคำสั่ง *void CBluetoothPMPEExampleEngine::DisconnectDevicesL();*

2. สเลฟ

สเลฟ เป็นอุปกรณ์ที่รอรับการเชื่อมต่อจาก มาสเตอร์ เพื่อสร้างเป็นเครือข่าย Piconet โดยในแต่ละ piconet จะมีเพียงหนึ่ง มาสเตอร์ ในโครงงานนี้สเลฟจะเป็นตัวที่ไปต่อกับวงจรควบคุมเครื่องใช้ไฟฟ้าอีกทีหนึ่ง โดยโปรแกรมในอุปกรณ์สเลฟจะทำงานดังนี้

- สเลฟ จะทำการเปิดพอร์ตรอรับการเชื่อมต่อจาก มาสเตอร์ โดยใช้คำสั่ง *TInt channel;*
channel = iListener->StartListenerL();

ซึ่งจะทำการรอรับฟังที่พอร์ต channel ที่ได้จากการหาพอร์ตที่ว่างของ Object iListener

- ขณะที่เซิร์ฟได้เปิดรอรับฟังการเชื่อมต่อจาก มาสเตอร์ เซิร์ฟเองก็จะทำการโฆษณา Service ออกไปทางพอร์ต channel เพื่อให้ มาสเตอร์ สามารถเชื่อมต่อ เข้ามาได้

iServiceAdvertiser->StartAdvertiserL(channel);

- เมื่อ มาสเตอร์ ทำการ Connect เข้ามาเรียบร้อยแล้ว จะมีฟังก์ชันมาจัดการเหตุการณ์นี้คือ

void CBluetoothPMPEngine::HandleListenerConnectedL();

- หากเซิร์ฟต้องการส่งข้อความไปยัง มาสเตอร์ ก็สามารถทำได้โดยเรียกใช้ฟังก์ชัน

void CBluetoothPMPEngine::SendMessage();

- ในกรณีที่เซิร์ฟได้รับข้อความจาก มาสเตอร์ จะมีฟังก์ชันมาจัดการเหตุการณ์นี้คือ

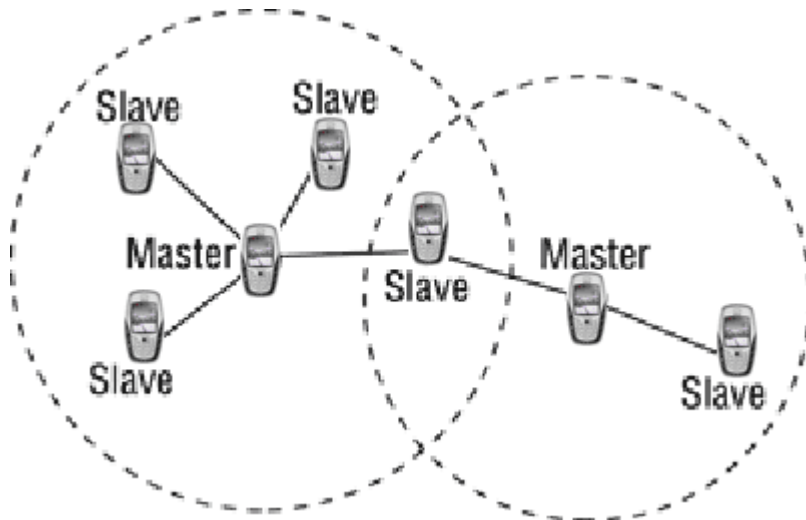
void CBluetoothPMPEngine::HandleListenerDataReceivedL(TDesC&aData)

เป็นฟังก์ชันสำคัญที่มีบทบาทในโครงงานนี้มาก ซึ่งจะได้กล่าวถึงต่อไป

- หากเซิร์ฟถูก ตัดการเชื่อมต่อจาก มาสเตอร์ เซิร์ฟก็จะไปเรียก *CBluetoothPMPEngine::HandleListenerDisconnectedL();*

เพื่อจัดการ จากนั้นก็จะทำการเปิดรอรับการเชื่อมต่อและโฆษณา Service อีกครั้ง

3.2.4 ศึกษาและเขียนโปรแกรมเพื่อทดสอบการติดต่อสื่อสารระหว่างโทรศัพท์เคลื่อนที่หลายเครื่องแบบ Point-to-Multipoint ผ่านการสื่อสารไร้สายแบบบลูทูธ ในเครือข่าย scatternet
ในขั้นตอนนี้เราจะนำโปรแกรม BluetoothPMP ที่ได้พัฒนาขึ้นมาในหัวข้อที่แล้วมาเพิ่มขีดความสามารถให้สามารถรองรับการเชื่อมต่อบนเครือข่าย Scatternet ของบลูทูธได้ ซึ่งเราได้พัฒนาโปรโตคอลในส่วนต่างๆ เพื่อใช้ในโครงงานนี้ ได้แก่



รูปที่ 3.12 เครือข่าย Scatternet

1. มาสเตอร์

สำหรับ มาสเตอร์ ใน เครือข่ายที่ออกแบบมานี้จะมีหน้าที่การทำงานเพิ่มขึ้นดังนี้

- เมื่อ มาสเตอร์ ได้รับข้อความคำสั่งจากเซลล์ที่ทำหน้าที่เป็น เกตเวย์ ก็จะทำให้การตรวจสอบว่า เป็นข้อความคำสั่งของ Piconet นั้นหรือเปล่า หากใช่ก็จะทำการส่งข้อความคำสั่งนั้นให้กับเซลล์ทุกตัว เพื่อทำงานต่อไป

```
void CBluetoothPMPEngine::HandleConnectorDataReceivedL(THostName
aName, TDesC& aData)
```

```
{
    ...
    // มาสเตอร์ sending data
    if(KMaxConnectedDevices)
    {
        for (TInt idx=0; idx<KMaxConnectedDevices; idx++)
        {
            if ( iConnectedDevices[idx]!=NULL )
            {
                THostName name;
                name=iConnectedDevices[idx]->iName;
                if(name!=aName)
                {
                    iConnectedDevices[idx]->SendDataL(msgtext8);
                }
            }
        }
    }
}
```

```

    }
}
}
}

```

- ถ้าหากไม่ใช่ มาสเตอร์ ก็จะส่ง ข้อความคำสั่งไปให้กับสเลฟที่ทำหน้าที่เป็นเกตเวย์ เพื่อส่งไปยัง Piconet ถัดต่อไป
- จากนั้นเราได้พัฒนา มาสเตอร์ ให้มี User Interface ที่สะดวกและสวยงามซึ่งเราได้พัฒนามาเป็น คอนโทรลเลอร์ ซึ่งรายละเอียดจะกล่าวในหัวข้อถัดไป
- นอกจากนี้เรายังได้พัฒนา มาสเตอร์ ให้เปิดพอร์ตรับการเชื่อมต่อเหมือนสเลฟในขณะที่ทำหน้าที่เป็น มาสเตอร์ ไปด้วย ทั้งนี้เนื่องจากต้องการให้ มาสเตอร์ สามารถรับคำสั่งจาก คอนโทรลเลอร์ ได้โดยตรง

2. สเลฟ

- นอกจากเมื่อสเลฟสามารถทำการเชื่อมต่อกับ มาสเตอร์ ในแต่ละ piconet แล้วสเลฟได้ถูกพัฒนาขึ้นให้สามารถเชื่อมต่อกับ มาสเตอร์ หลายตัวได้ ซึ่งสเลฟตัวนี้พัฒนาขึ้นมาทำหน้าที่เป็น เกตเวย์ ต่อไป โดยมีหลักการคือ เมื่อสเลฟทำการเชื่อมต่อกับ มาสเตอร์ แล้วสเลฟนั้นจะเปิดพอร์ตใหม่เพื่อรับการเชื่อมต่อจาก มาสเตอร์ ตัวอื่น หรือ คอนโทรลเลอร์ โดยตรง

```

void CBluetoothPMPEngine::HandleListenerConnectedL()
{
    ShowMessageL(_L("Connected!\n"), true);
    if (สเลฟState==0)
    {
        iListener1 = CListener::NewL(this, &iSocketServ);
        StartสเลฟLL();
    }
    สเลฟState++;
}

```

- สเลฟ แต่ละตัวจะเก็บคำสั่งของตัวเองไว้เมื่อได้รับข้อความคำสั่งก็จะทำการเปรียบเทียบว่าข้อความนั้นตรงกับที่เก็บไว้หรือไม่ หากใช่ก็จะทำการ

ประมวลผลส่งสัญญาณด้วยเสียงความถี่ต่างๆ ไปยังวงจรควบคุมอุปกรณ์
ไฟฟ้าที่ต่ออยู่

```
void CBluetoothPMPEngine::HandleListenerDataReceivedL(TDesC& aData)
{
    ...

    TBuf<80> command1;
    command1.Format(_L("Restroom_TV_On"));

    TBuf<80> command2;
    command2.Format(_L("Restroom_TV_Off"));

    ...

    if (msg2==command1)
    {
        iMdaAudioToneUtility->PrepareToPlayTone(500,
TTimeIntervalMicroSeconds(KSoundDuration));

        PlayL();
    }
    else if (msg2==command2)
    {
        iMdaAudioToneUtility->PrepareToPlayTone(530,
TTimeIntervalMicroSeconds(KSoundDuration));

        PlayL();
    }
    ...
}
```

- เสลฟ สามารถรับการเชื่อมต่อจาก คอนโทรลเลอร์ โดยตรงได้ ซึ่งเมื่อกดเลิก
การเชื่อมต่อแล้วเสลฟก็จะกลับไปอยู่ในสถานะรอรับฟังการเชื่อมต่อใหม่
ต่อไป

3. คอนโทรลเลอร์

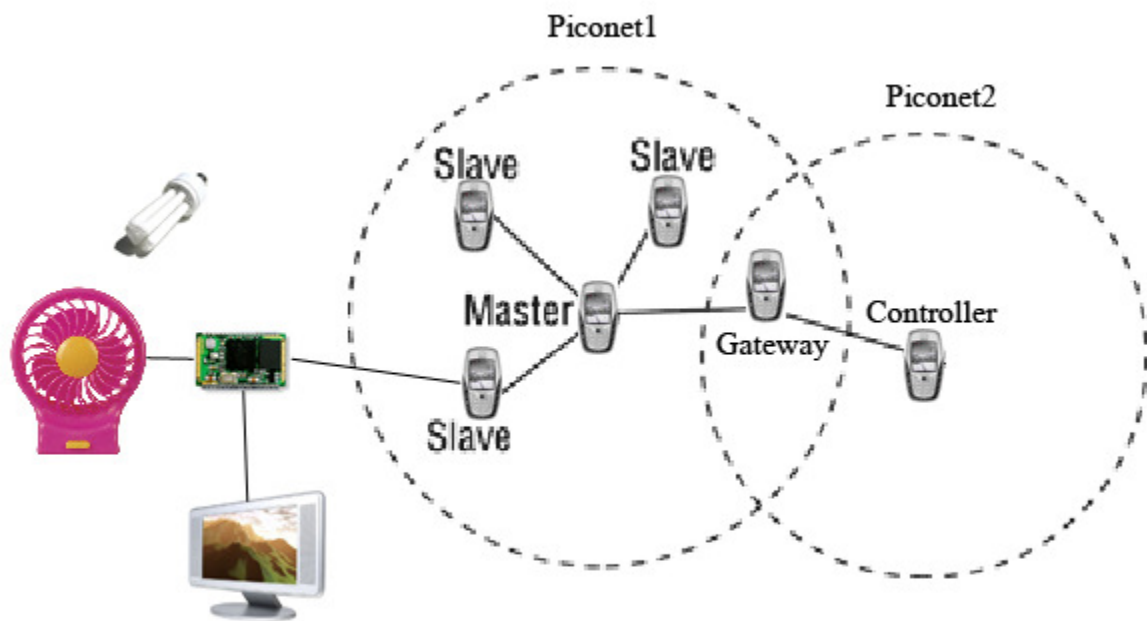
- คอนโทรลเลอร์ คือ มาสเตอร์ ที่พัฒนาขึ้นมาให้มี User Interface ที่สวยงาม และมีคุณสมบัติเพิ่มขึ้นมาด้วย ได้แก่

1. User Interface ได้พัฒนาเป็นเมนู ดังภาพ



รูปที่ 3.13 User Interface ของ คอนโทรลเลอร์

2. คอนโทรลเลอร์ ได้พัฒนาให้สามารถเชื่อมต่อเข้ากับ มาสเตอร์ หรือเสลฟใน Piconet อื่นได้ โดยมีเงื่อนไขดังนี้
 - ถ้า คอนโทรลเลอร์ อยู่ในระยะที่สามารถเชื่อมต่อกับ มาสเตอร์ ได้ คอนโทรลเลอร์ ก็จะทำการเชื่อมต่อกับ มาสเตอร์ ที่อยู่ใกล้ที่สุด
 - ถ้า คอนโทรลเลอร์ ไม่อยู่ในระยะที่สามารถเชื่อมต่อกับ มาสเตอร์ ได้ คอนโทรลเลอร์ ก็จะทำการเชื่อมต่อกับเสลฟที่อยู่ใกล้ที่สุด ซึ่งเสลฟตัวนั้นจะกลายเป็น เกตเวย์ และ คอนโทรลเลอร์ ก็จะทำหน้าที่เป็น มาสเตอร์ ของอีก piconet หนึ่ง



รูปที่ 3.14 การเชื่อมต่อของ คอนโทรลเลอร์ กับระบบ
โดยโคดส่วนที่ คอนโทรลเลอร์ ใช้ในการเชื่อมต่อคือ

```
void CBluetoothPMPEngine::ConnectDevicesL()
{
    ...
    CConnector *connector = NULL;
    for ( TInt idx=0; idx<(iDevDataList.Count()); idx++ )
    {
        if ( iConnectedDeviceCount>=KMaxConnectedDevices )
            return;

        TDeviceData *dev = iDevDataList[idx];
        connector = CConnector::NewL(this, &iSocketServ);
        if ( มาสเตอร์==dev->iDeviceName )
        {
            if ( dev->iDeviceServicePort>0 )
            {
                if ( (connector->ConnectL(dev->iDeviceName, dev->iDeviceAddr, dev-
                >iDeviceServicePort))==KErrNone )
```



```

        {
            iConnectedDevices[iConnectedDeviceCount] = connector;
            iConnectedDeviceCount++;
        }
        else
        {
            delete connector;
        }
    }
}

if (!HasConnections())
{
    TDeviceData *dev = iDevDataList[0];
    connector = CConnector::NewL(this, &iSocketServ);
    if ( dev->iDeviceServicePort > 0 )
    {
        if ( (connector->ConnectL(dev->iDeviceName, dev->iDeviceAddr, dev-
>iDeviceServicePort)) == KErrNone )
        {
            iConnectedDevices[iConnectedDeviceCount] = connector;
            iConnectedDeviceCount++;
        }
        else
        {
            delete connector;
        }
    }
}

ShowConnectedDevicesL();
}

```

- เมื่อ คอนโทรลเลอร์ ทำการเชื่อมต่อกับ มาสเตอร์ หรือสเลฟ(เกตเวย์) แล้ว คอนโทรลเลอร์ ก็จะส่งข้อความเพื่อควบคุมอุปกรณ์ไฟฟ้า โดยรูปแบบคำสั่งมีดังนี้

Bedroom_TV_On

Bedroom เป็นชื่อของสเลฟตัวที่ต้องการควบคุม

TV เป็นชื่อของอุปกรณ์ที่ต้องการควบคุม

On เป็นสิ่งที่ต้องการให้อุปกรณ์นั้นๆทำ

- เมื่อยกเลิกการเชื่อมต่อแล้ว ก็สามารถทำการเชื่อมต่อได้ใหม่ เนื่องจาก มาสเตอร์ และสเลฟจะทำการรอรับฟังการเชื่อมต่อไว้ตลอด

4. เกตเวย์

เกตเวย์ คือสเลฟที่ถูกพัฒนาขึ้นมาเพื่อรับส่งข้อความคำสั่งระหว่าง piconet (มาสเตอร์-มาสเตอร์ หรือ มาสเตอร์-คอนโทรลเลอร์) ดังนั้น เกตเวย์ จึงสามารถต่อกับวงจร และส่งสัญญาณให้วงจรได้เหมือนสเลฟทุกประการ

3.2.5 สร้างวงจรควบคุมอุปกรณ์ไฟฟ้าและการเปิดปิดอุปกรณ์ไฟฟ้า

ในขั้นตอนนี้เราจะสร้างวงจรควบคุมอุปกรณ์ไฟฟ้าและการเปิดปิดอุปกรณ์ไฟฟ้า โดยใช้วงจรจะรับสัญญาณ input จากตัวสเลฟโดยเมื่อสเลฟได้รับคำสั่งจาก มาสเตอร์ หรือ คอนโทรลเลอร์ ก็ปล่อยสัญญาณเสียงความถี่ต่างๆ ตามแต่ละคำสั่งเพื่อป้อนให้กับวงจรที่มี MCS-51 เพื่อประมวลผลและควบคุมอุปกรณ์ไฟฟ้าต่อไป โดยโปรแกรมที่เขียนใน MCS-51 คือ

```
//freq.c
```

```
#include <REG2051.H>
```

```
#include<stdio.h>
```

```
#include<intrins.h>
```

```
sbit start = P1^7;
```

```
sbit out1 = P1^0;
```

```
sbit out2 = P1^1;
```

```
sbit out3 = P1^2;
```

```
sbit out4 = P1^3;
```

```
sbit out5 = P1^4;
```

```
sbit out6 = P1^5;
```

```
sbit out7 = P1^6;
```

```
sbit out8 = P1^7;
```

```
unsigned long  temp1;
```

```
unsigned long  temp2;
```

```
unsigned int    T,freq;
```

```
void demsec (unsigned int count)
```

```
{
```

```
    unsigned char i;
```

```
    while(count)
```

```
    {
```

```
        for(i=1;i<152;i++);
```

```
        count--;
```

```
    }
```

```
}
```

```
void main(void)
```

```
{
```

```
    SCON=0x52;  // rs232
```

```
    TMOD=0x22;  // auto reload timer 0
```

```
    TH1=0xfd;    //11.059
```

```
    TR1=1;
```

```
//-----int timer0 set -----
```

```
    TH0=0x00;  // 100 usec  11.059 * 2
```

```
    TL0=0x00;
```

```
    TR0=1;
```

```

ET0=0;
//----- int EX0 --P3.2-----
EX0=1;
IT0=1;
EA=1; //----int all

while(1)

{

    temp2 = temp2 * 0x100;
    T = temp2 | temp1;
    freq=1/T;
    demsec(200);

    if((freq>500)&(freq <510)) {out7=0;}
    if((freq>530)&(freq <540)) {out7=1;}

    if((freq>560)&(freq <570)) {out6=0;}
    if((freq>580)&(freq < 590)) {out6=1;}

    if((freq>600)&(freq < 610)) {out5=0;}
    if((freq>620)&(freq < 630)) {out5=1;}

    if((freq>640)&(freq <650)) {out4=0;}
    if((freq>660)&(freq <670)) {out4=1;}

    if((freq>680)&(freq < 690)) {out3=0;}
    if((freq>700)&(freq < 710)) {out3=1;}

    if((freq>720)&(freq < 730)) {out2=0;}
    if((freq>740)&(freq < 750)) {out2=1;}

```

```

        if((freq>760)&(freq < 770)) {out1=0;}
        if((freq>780)&(freq < 790)) {out1=1;}

        if((freq>800)&(freq < 810)) {out8=0;}
        if((freq>820)&(freq < 830)) {out8=1;}

    }

}

// int t0-----

void Ex0(void) interrupt 0
{

    round_count_low=TL0;
    round_count_high=TH0;
    TL0=0x00;
    TH0=0x00;
}

//--int1 -----

void timer0(void) interrupt 1
{

}

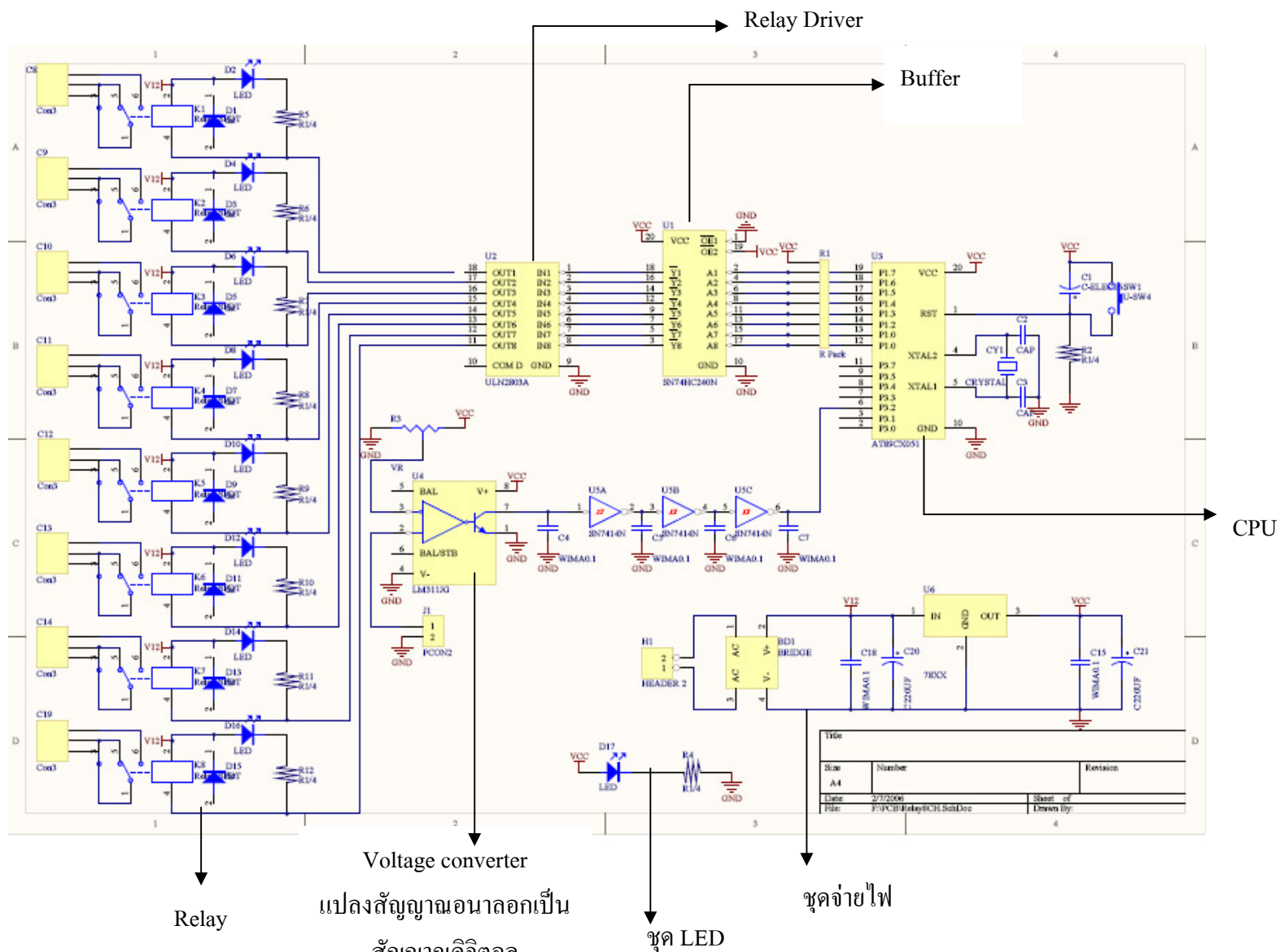
```

จากโปรแกรมจะเห็นได้ว่าเมื่อวงจรได้รับความถี่ก็จะตรวจสอบว่าอยู่ในช่วงความถี่ไหน จากนั้นจึงไปควบคุม output ตัวนั้นๆ

รายการอุปกรณ์ที่ใช้

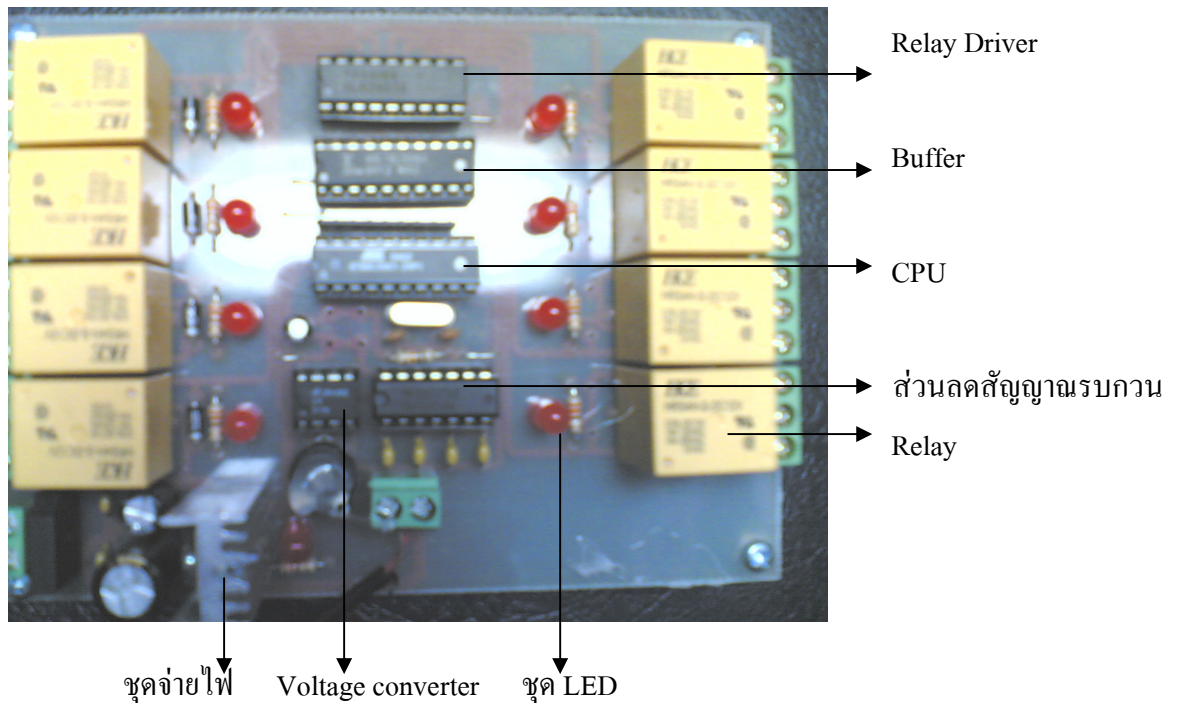
- | | | |
|----|-----------------------|-------|
| 1. | Relay 12 V | 1 ตัว |
| 2. | Relay Driver ULN2803A | 1 ตัว |
| 3. | Buffer SN74HC240N | 1 ตัว |

4.	MCS-51 AT89CX051	1 ตัว
5.	OpAmp LM311JG	1 ตัว
6.	Smith-Trigger SN7414N	1 ตัว
7.	Crystal	1 ตัว
8.	Resistor	12 ตัว
9.	Capacitor	14 ตัว
10.	Adapter 12 V 1.2A	1 ตัว

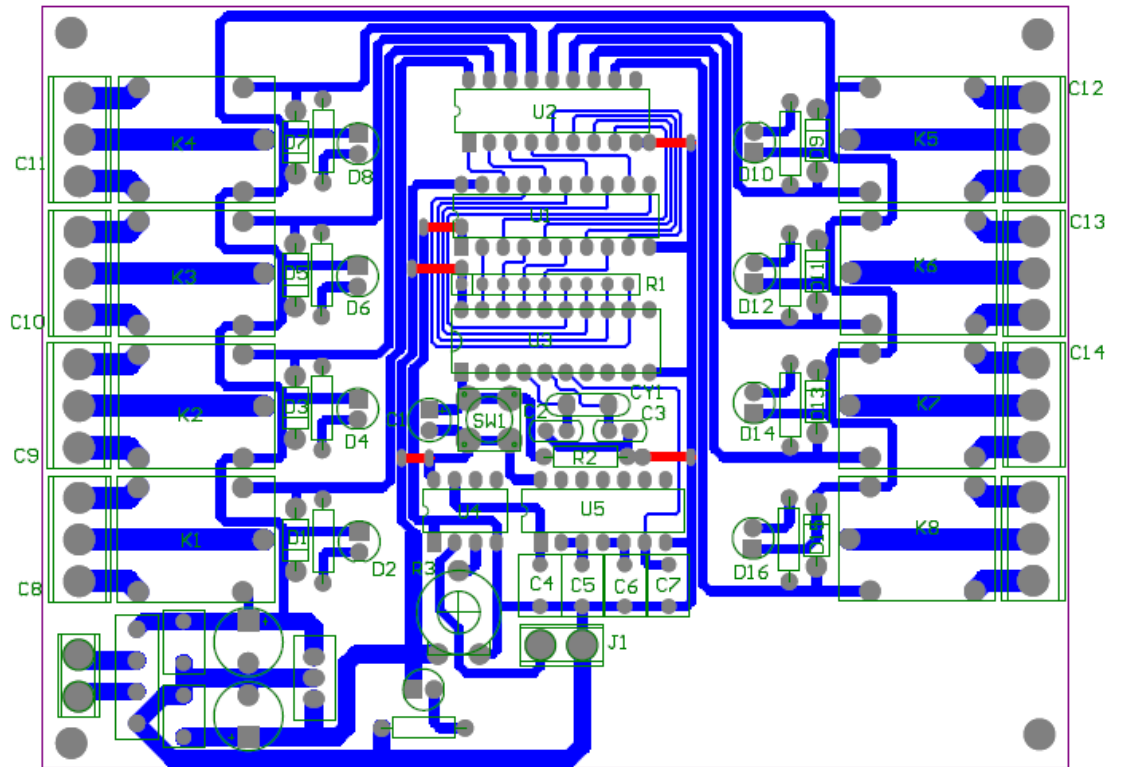


รูปที่ 3.15 ภาพวงจรที่ได้นำมาแปลงสัญญาณความถี่ให้เป็นสัญญาณ

จากรูปที่ 3.14 เมื่อสัญญาณความถี่ (สัญญาณอนาลอก) ที่ได้รับมาจากมือถือจะเข้า LM311JG voltage converter แล้วจะได้เป็นสัญญาณดิจิทัลแบบ TTL จากนั้นจะผ่านส่วนที่จะทำการลดสัญญาณรบกวน (noise) แล้วเข้าในส่วนของตัวประมวลผล ซึ่งจะมีการคำนวณหาความถี่โดยจะจับเวลาระหว่างช่วงของสัญญาณขาออกจากสูตร $f = 1/T$ หลังจากนั้นสัญญาณจะผ่าน Buffer และ Relay Driver แล้วไปแสดงผลในส่วนสุดท้าย



รูปที่ 3.16 ภาพวงจรจริงและส่วนประกอบต่างๆของวงจร



รูปที่ 3.17 แสดงเส้นของแผ่นปริ้นท์